Week-3 (Python)

- while (t-->0){ } Not valid in python

  }

  (t--) → no syntax in Python, do it normally

- For loop each → For x in num{

  }

  For loop → For i in range (0, 10, 2){

  initial point, end point, step

  }

F-strings → String formatting methods

```
Print ( f"{num} x {i} = {num * i}")
```

Decimal formating

```
x = 22/7
Print ( f"{x:.3f}")   → 3.141
```

(Precision) → precision

Structore: [Fill] [width] .[Precision]

(width) → min. space reserved for the values.

```
a = 5
b = 10
c = 15
print (f"{a:5}{b:5}{c:5}")
```

.....5 ___ 10 ____ 15

(width + precision)

```
print (f"{Pi: 10.3f}")   → ------ 3.143
```

Fill Characters → Fill empty space with characters

num = 25

Print(f"{num:06}") → 000025 — Left align

Print(f"{num:*<6}") → 25**** — Right align

Print(f"{num:*^6}") → **25** — Center align

ex:
```
for i in range(1,6):
    print(f"{2:3} x {i:3} = {2*i:3}")
```

output:

```
  2 x   1 =  2
  2 x   2 =  4
  2 x   3 =  6
  2 x   4 =  8
  2 x   5 = 10
```

- Print() → newline
- Print("Welcome", end="") → Welcome Python
  Print("Python")

sep → Print('G','F','E', sep='') → GFE

Print('09','12','2016', sep='-') → 09-12-2016

---

break → stop loop immediately

```
for i in "Hello @world":
    if i == '@':
        break
    print(i)
```
→ Hello

Continue → skips current iteration.

```
for x in "hello":
    if x == 'l':
        continue
    print(x)
```
→ h e o

Pass → Does nothing (placeholder)

```
if 5==5:
    pass
else: print()
```
→ code block required syntactically but no action needed

Integer.MIN_VALUE — Python → -float('-inf')

max — Py → float('inf')

WEEK-4

LISTS

- Python lists → ordered
  → Mutable
  → Indexed
  → allows duplicate values
  → can store multiple datatypes

list[0] → first ele.
list[-1] → last ele

⇒ Listslicing x = l[2:5] y = l.[:4]
Print(x, y)
↳ output : [3,4,5] [1,3,3,4]

⇒ adding items to list :
↳ l.append(8)

⇒ insert() → adds element at specific index.
⇒ List.insert(index, value)

⇒ l.pop() → remove last ele.
l.pop(i) → remove a specific index
l.remove(3) → remove a specific value.
l = [2, 3, 4, 5]
★ del l[2] → deletes element by idx
len(l) → length of list

l = [1, 2]
l.extend([3,4])
Print(l)
↳ [1,2,3,4]
l.extend() → for adding multiple items

- l[3] = 9 → reassignment possible (l[3] ele., List)
print('dog' (in) pet) → Check if item exists (return Boolean value)
↳ operator

- extend() (merges list) →
l1 = [1,2,3]
l2 = [4,5,6]
l1.extend(l2) → 1 2 3 4 5 6

- Sorting Lists using fone.
L = [5,2,9,1,3]
ascending: L.sort() → [1,2,3,5,9]
↳ return void
L.reverse()
descending: L.sort(reverse = True)

Matrix Multiplication → normal iteration approach:

```
for i in range(dim)
    for j ---
        for k ---
            C[i][j] += A[i][k] * B[k][j]
```

Numpy shortcut:

```
import numpy
x = numpy.mat(A)
y = numpy.mat(B)
print(x * y)
```

Recursion
Func. application

WEEK-5

Functions → def function_name(parameters):
statements

⇒

```
def square(x):
    return x*x
print(square(4)) #16
```

Scope → Local Scope → inside func. Not accessible outside
→ Global Scope → outside defined func. available everywhere

Modifying global variable

```
x=5
def change():
    global x   <-- to avoid unbound local error
    x = x+10
change()
print(x) # 15
```

Function Arguments

Positional Arguments

```
def add(a,b):
    return a+b
print(add(2,3))
```

Default Argument

```
def add(a, b=10):
    return a+b
print(add(5)) #15
print(add(5,3)) #8
```

Keyword Arguments

```
def add(a,b):
    return a+b
print(add(b=5, a=3))
```

Mixing Rule:

add(3, b=5) ✓
add(a=3, 5) ✗ error

WEEK-6

Python Collections
- List
- Tuples
- Sets
- Dictionary

1) Copying

```
a = [1,2,3]
b = a
b[0] = 100
print(a)
↳ [100,2,3]
```

```
a = [1,2,3]
b = a.copy()
b[0] = 100
print(a) → #[1,2,3]
```

2) Tuples
- Ordered
- Immutable
- Allows duplicate

```
t = (1,2,3)
```

Single element tuple → (5,)
↳ comma is imp.

- accessing:

```
t = (10,20,30)
print(t[1]) #20
t[0] = 10 x error
```

- Lists are mutable inside tuple

```
t = (1,2,[3,4])
t[2][0] = 100.
↳ print(t) = (1,2,[100,4])
```

Methods:

```
t = (1,2,2,3)
print(t.count(2))  # 2
print(t.index(3))  # 3
```

3) SETS
- Unordered
- no duplicates
- Mutable

```
s = {1,2,2,3} → Print: {1,2,3}
s = set()  → empty set
```

- add ele. → s.add(3)
- s.update([4,5])
- s.remove(2) # error if not present
- s.discard(10) # no error (safer) → to use if not sure if element exist
- s.pop() → remove random elements

```
A = {1,2}
B = {2,3}
print(A|B) → {1,2,3}
  (Union)
print(A&B) → {2}
  (intersection)
print(A-B) → #1
  (difference)
```

- Symmetric difference
  - ↳ Print (A ^ B) → # {1, 3}

A = {1, 2}
B = {2, 3}

---

④ DICTIONARY → key-value pair like HashMap

- ↳ keys → unique
  values → can repeat
  keys → immutable
  mutable structure

d[i] ⇒ items[i]
- key
- ↳ for putting values

- access: d = {"name": "Jack", "age": 26}
  print (d["name"]) # Jack
  print (d.get("age")) #26

- Remove
  d.pop("age")
  d.popitem() # removes last
  d.clear() # remove all

dictionary methods:

d.keys()
d.values()
d.items()

- looping
  for key ind:
  print (key, d[key])

- membership
  Print("name" ; nd) #True

---

checking membership universal
- not string[s] in alphabets
- string[i] in alphabets

Course Count Problem (Data Processing - 2) Function Type

def courses_sorted_by_enrollment (student_courses: dict) → list:

course_count = {}

// Count Enrollments
for courses in student_courses.values():
  for coure in courses:
    course_count [course] = course_count.get (course, 0) + 1

// Sorted courses by count (descending)
sorted_courses = sorted (course_count, key = lambda x: course_count [x], reverse = True)
return sorted_courses

WEEK-4

→ Comments (#)

(1) Syntax and Indentation
↳ Py. uses indentation (spaces/tabs) to define blocks

✓ if x > 4:
    Print("Hello")

✗ if 5>4:
x = "Hello"
print(x)

(2) • print func. → Print("Hello") → Placeholder
• user input → name = input("Entername:")
Print(name)

input → always return string

For datatypes, use type casting,
age = int(input("Enterage:"))
Price = float(input("Enter price:"))

(3) • Boolean values are capital first → True, False
• checking Type → ex: x = 5
Print(type(x)) → # int
• Type casting: str(5) → "5"
int("5") → 5
float(4) = 4.0
bool(0) → false
bool(10) → True
bool("") → false

(4) Arithmetic Operators:

/ → divide (float result always)	5/2 = 2.5
// → floor division	5//2 = 2
** → Power	2**3 = 8

Associativity
when same priority of operator appear → direction matter,
ex: 10-5-2 → (10-5) - 2 = (3) (left → right)
2**3**2 → 2**(3**2) = 2**9 = 512
(right → left)

(5) Logical Operators → and, or, not

print(not(false)) → True
print(True and false) → false

"True" or "false"
↳ str.

(6) Strings

' ' " " → normal str.
''' ''' → multi-line str
""""" """"" → multi-line str.

Accessing parts of a string → text = "Hi Everyone"
Print(text[0]) → H

[Slicing]
→ str[start:end:stop] (end → exclusive)
text = "Python Lower"
Print(text[2:5])

-1 → last char.
-2 → 2nd last char

# Replication
text = 'Gagan'
x = text * 4
Print(x) → GaganGaganGaganGagan

text = 'Hello'
Print(len(text)) → 5
Use pop() → to delete idx

# String Comparison
x = 'India'
Print(x == 'India') → True
('ab' < 'az') → True
('abcdef' < 'abcde') → False

String Methods (Built-in)
1) .upper() / .lower() → converts case
2) .capitalize() → Capitalizes only the 1st char.
3) title() → Capitalizes 1st char of every word
4) strip() → .lstrip() → left
   → .rstrip() → right
5) .isdigit() → returns (true) if all characters are digits
6) case check → .islower(), .isupper()
7) .count('x') → find occurences
   .index('x')
   .replace('old', 'new')

ESCAPE CHARACTERS
⇒ When using double quotes inside quotes → "We are "Vikings"" → error
⇒ \" Vikings \" → " Vikings "
⇒ \\ → \
⇒ \n → newline
⇒ \t → tab space
→ del(z) → delete var.

WEEK-2

- **Assignment-Operators:**

Assignments

1) Direct : x, y = 1, 2 → x = 1, y = 2

2) Chained: x = y = z = 8 → x = 8, y = 8, z = 8

3) Unpacking: a, b = 100, 200, 300 ✗
or a, b, c = 100, 200 ✓

a, b = "56" → a = '5', b = '6'

So unpack:

★ a, *b = 100, 200, 300 → a = 100 (int), b = [200, 300] (List)

*a, b → a = [100, 200] (List), b = 300 (int)

- **Operators** Shorthand: x += 3 ≈ x = x + 3

Membership: in / not in

ex: sentence = "Gagneet Kaur"

print ("Kaur in sentence") #Returns True

x--
x++
(not valid)

- **Conditionals** → · If/Else
  · Elif
  → Ternary: Print ("Even") if n%2 == 0 else print ("odd")

- Module & Libraries → import math → math.sqrt()
  math.pow()
  math.ceil()
  math.floor()
  math.pi()
  math.cos(x)

  abs(x-y)

⇒ random module → · random()
· randint(x, y)

⇒ calender module → import calendar
→ print (calendar.month (2020, 10))

WEEK-8 (Recursive Programs)

① Find 0 in a list using Recursion

```
def check0(L):
    if (len(L) == 0):
        return False
    if (L[0] == 0):
        return True
    else:
        return check0(L[1 : len(L)])
```

② Sorting Recursively → 1. find min
2. Remove it
3. Recursively sort rest

```
def find_min(lst):
    mini = lst[0]
    for x in lst:
        if x < mini:
            mini = x
    return mini

def recursive_sort(lst):
    if len(lst) <= 1:
        return lst
    m = find_min(lst)
    lst.remove(m)
    return [m] + recursive_sort(lst)
```

ex:

```
lst = [5, 6, 59, 19, 2, 7]
print(recursive_sort(lst))
```

output:

[2, 5, 6, 7, 19, 59]

③ Binary Search

Iterative: →

```
def binary_search(lst, k):        # k → element to be searched
    begin = 0
    end = len(lst) - 1
    while begin <= end:
        mid = (begin + end) // 2
        if lst[mid] == k:
            return True
        elif lst[mid] > k:
            end = mid - 1
        else:
            begin = mid + 1
    return False
```

- Recursive Binary search :→

```
def recursive_binary_search(lst, k, begin, end):
    if begin > end:
        return False
    mid = (begin + end) //2
    if lst[mid] == k:
        return True
    elif lst[mid] > k:
        return recursive_binary_search(lst, k, begin, mid-1)
    else:
        return recursive_binary_search(lst, k, mid+1, end)
```

WEEK-9 (File Handling)

```
f = open("data.txt", "r")      #open
data = f.read()                #operation
f.close()                      #close
```

Opening files in Python

f = open(filename, mode)

Five modes:

- r → Read (default)
- w → write (orerwrite)
- a → append
- b → binarymode

Text Mode → returns strings
Binary mod → returns bytes

ex: f = open("image.jpg", "rb")

Closing files

⇒ Safe way (Imp):

```
try:
    f = open("test.txt", "r")
    data = f.read()
finally:
    f.close()
```

⇒ Best Method. (auto close)

```
with open("test.txt", "r") as f:
    data = f.read().
```

Writing to files

- use 'w' mode
- overwrites existing file
- creates file if not exists

```
f = open("test.txt", "w")
f.write("my file\n")
f.write("I am Gagneet")
f.close()
```

write() does not add newline automatically use \n

Reading files

```
f = open("test.txt", "r")
print(f.read(4))  # first 4 chars.
print(f.read(4))  # next 4 chars.
print(f.read())   # rest of file
f.close()
```

Each read continues from where previous read left off.

returns "" when file ends
\n is included in output

File Pointer

Methods: → `f.tell()` # current position
`f.seek(0)` # move pointer.

Reading Line by Line

```
f = open("test.txt", "r")
print(f.readline())
print(f.readline())
f.close()
```

Big File Processing → Logic: • Read Line by Line • Stop when found

```
code → f=open("directory.txt","r")
       flag= false
       while True:
           line = f.readLine()
           if line = "":
               break
           num =int(line.strip());
           if num == 9024876051:
               print("The no. was found")
               flag = True
               break
       if not flag:
           Print("The no. was not found")
       f.close()
```

Caesar Cipher (Encryption) → Concept:
- Shift letters by 3
- a → d, b → e

```
import string
def create_caesar_dictionary():
    letters = string.ascii_lowercase
    d={}
    for i in range(len(letters)):
        d[letters[i]] = letters[(i+3)%26]
    return d
f =open("sherlock.txt","r")  #open already existing file
f = open("encrypted-Sherlock.txt",'w')  # creates new file
d = create_caesar dictionary()  #creates a dictionary
```

```
c = f.read(1)          # reads one character from file f.
while (c != ''):       # end of file condition
    g.write(d[c])      # writes in file g
    c = f.read(1)      # reads next character in file f.

f.close()
g.close()
```

#Genetic Sequence Problem → check if a sequence exists

```
f = open("human.txt", "r")
sequence = f.read()
bp = "AATCGAA"
if bp in sequence:
    print("sequence found")
else:
    print("sequence not found")
f.close()
```

WEEK-10 OOPS

- class: ; Blue Print / Template
  - Defines structure + behaviour
- object:
  - Instance of class
  - Contains real data

Ex:

```
class Test:
    def fun(self):
        Print("Hello")

obj = Test()      # Object creation
obj.fun()         # method call
```

this is optional in Java when name clash

in python self is mandatory.

⇒ The self keyword

- self refers to current obj.
- Always 1st parameter in methods

```
class Student:
    def show(self, name):
        Print("Name:", name)
s = Student()
s.show("Yash")
```

Constructor __init__ → run automatically when obj. created
↳ special method

```
class Person:
    def __init__(self, name):
        self.name = name

    def say_hi(self):
        print("Hello, my name is:", self.name)

p = Person("Jack")
p.say_hi()
```

instance variable

don't use normal local variable can't be accessed out

Output: Hello, my name is Jack

Class vs Instance var.

Shared amongall objects

→ unique for each object

```
class CSStudent:
    stream = "CSE"          # class variable

    def __init__(self, roll):
        self.roll = roll    # instance var.

a = CSStudent(101)
b = CSStudent(102)
print(a.stream)   # CSE
print(b.stream)   # CSE
print(a.roll)     # 101
```

Inheritance & Method Overriding

```
p = Penguin()
├── Penguin.__init__()
│   ├── super().__init__()  → call parent constructor
│   │   └── Bird.__init__() → "Bird is ready"
│   └── print → "Penguin is ready"
├── p.whoIsThis() → "Penguin" (overridden)
├── p.swim()      → "Swim faster" (inherited)
└── p.run()       → "Run faster" (child method)
```

```
class Bird:
    def __init__(self):
        Print("Bird is ready")

    def whatIsThis(self):
        Print("Bird")

    def Swim(self):
        Print("Swim faster")

class Penguin(Bird):
    def __init__(self):
        super().__init__()
        Print("Penguin is ready")

    def whoIsThis(self)
        Print("Penguin")

    def run(self):
        Print("Run faster")

P = Penguin()
P.whoIsThis()
P.Swim()
P.run()
```

OUTPUT:

```
BIRD is ready
Penguin is ready
Penguin
Swim faster
Run faster
```

Data Hiding (Encapsulation) → restrict direct access to variables

```
class MyClass:
    def __init__(self):
        self.__hidden = 0  # private variable
    def add(self, x):
        self.__hidden += x
        Print(self.__hidden)

Obj = MyClass()
Obj.add(2)
Obj.add(5)
# Print(obj.__hidden) # ERROR (AttributeError)
```

Output:

```
2
7
AttributeError
```

Numpy

Feature	List	Numpy
Type	Mixed	Same
Speed	Slow	Fast
Memory	More [X]	Less [✓]
Math ops		

- creating arr.

```
import numpy as np
arr = np.array([1,2,3,4])
print(arr)
print(type(arr)) ⟶ <class 'numpy.ndarray'>
```

- dimensions of arr.

```
a = np.array(42)              # 0D ⟶ a.ndim = 0D
b = np.array([1,2,3])         # 1D ⟶ b.ndim = 1D
c = np.array([[1,2],[3,4]])   # 2D ⟶ c.ndim = 2D
```

Matplotlib

```
import matplotlib.pyplot as plt
```

⇒ **Basic Plots**

(1) Scatter Plot

```
x = np.array([1,2,3,4])
y = np.array([10,20,30,40])
plt.scatter(x,y)
plt.show()
```

(2) Bar Chart

```
x = np.array(["A","B","C"])
y = np.array([3,8,1])
plt.bar(x,y)
plt.show()
```

(3) Pie Chart

```
y = np.array([35,25,25,15])
labels = ["Apple","Banana","Cherry","Dates"]
plt.pie(y, labels = labels)
plt.show()
```

(4) SubPlot → multiple plots in one figure

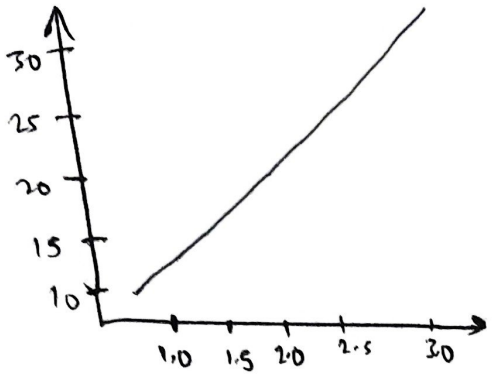
```
x = np.array([1,2,3])
y = np.array([10,20,30])

plt.subplot(2,2,1)
plt.plot(x,y)
plt.subplot(2,2,2)
plt.plot(y,x)
plt.show()
```

Positions:

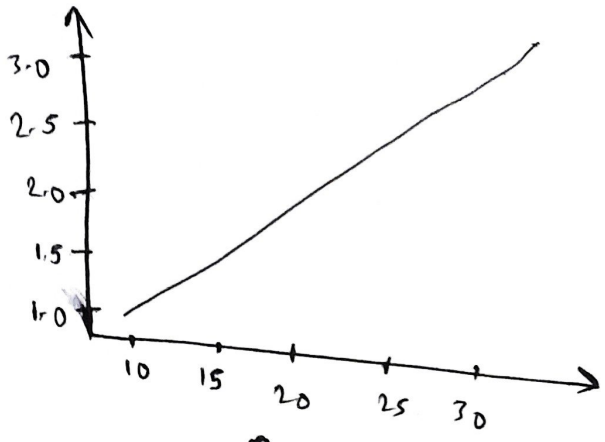
[1] [2]

[3] [4]

(2, 2, 1) → row, col, position



(2, 2, 1)



(2, 2, 2)

(Rows-cols swapped)